

```
path = Path('alice.txt')
count_words(path)
```

Most of this code is unchanged. It's only been indented, and moved into the body of `count_words()`. It's a good habit to keep comments up to date when you're modifying a program, so the comment has also been changed to a docstring and reworded slightly ❶.

Now we can write a short loop to count the words in any text we want to analyze. We do this by storing the names of the files we want to analyze in a list, and then we call `count_words()` for each file in the list. We'll try to count the words for *Alice in Wonderland*, *Siddhartha*, *Moby Dick*, and *Little Women*, which are all available in the public domain. I've intentionally left *siddhartha.txt* out of the directory containing *word_count.py*, so we can see how well our program handles a missing file:

```
from pathlib import Path

def count_words(filename):
    """Count the words in a file."""
    filenames = ['alice.txt', 'siddhartha.txt', 'moby_dick.txt',
                 'little_women.txt']
    for filename in filenames:
        ❶ path = Path(filename)
        count_words(path)
```

The names of the files are stored as simple strings. Each string is then converted to a `Path` object ❶, before the call to `count_words()`. The missing *siddhartha.txt* file has no effect on the rest of the program's execution:

```
The file alice.txt has about 29594 words.
Sorry, the file siddhartha.txt does not exist.
The file moby_dick.txt has about 215864 words.
The file little_women.txt has about 189142 words.
```

Using the `try-except` block in this example provides two significant advantages. We prevent our users from seeing a traceback, and we let the program continue analyzing the texts it's able to find. If we don't catch the `FileNotFoundError` that *siddhartha.txt* raises, the user would see a full traceback, and the program would stop running after trying to analyze *Siddhartha*. It would never analyze *Moby Dick* or *Little Women*.

Failing Silently

In the previous example, we informed our users that one of the files was unavailable. But you don't need to report every exception you catch. Sometimes, you'll want the program to fail silently when an exception occurs and continue on as if nothing happened. To make a program fail silently, you write a `try` block as usual, but you explicitly tell Python to do